Operations Research Project

Team Members:-

- Alen Binoj MDL24CSBS012

A. Problem Definition

A1. Project Title

Cost-Optimal GPU Provisioning and Workload Allocation for AI Inference Using Operations Research

A2. Problem Statement

An enterprise operates multiple internal AI services, including customer support chatbots, document search and summarization systems, and offline code-audit batch jobs. These services use open-source large language models deployed across heterogeneous cloud GPU instances, such as NVIDIA T4, A10G, A100, and H100 GPUs. Each GPU type differs in hourly rental cost, video memory (VRAM) capacity, memory bandwidth, and inference throughput.

The demand for these AI services fluctuates over time, while individual requests vary in context length, processing requirements, and latency deadlines. Maintaining excessive GPU capacity can result in unnecessary cloud expenditure, whereas insufficient capacity or inefficient workload allocation can cause processing delays, memory limitations, and service-level agreement (SLA) violations.

The problem is to develop an Operations Research optimization model that determines the number and types of GPU instances to activate during each planning interval and allocates workloads to suitable GPU types. The objective is to minimize total GPU operating costs while satisfying workload demand, GPU memory and processing capacity constraints, throughput requirements, and latency deadlines represented in the model.

The proposed solution will be implemented using Python and Mixed-Integer Linear Programming (MILP), with the PuLP library and CBC solver. The model's performance will be evaluated using numerical results, comparisons with a baseline allocation strategy, and sensitivity analysis under different workload and cost scenarios.

A3. Real-World Context

Large language models are increasingly used in enterprise applications such as customer support automation, knowledge retrieval, document summarization, and software code analysis. Deploying these models for inference requires substantial computational resources, often provided through cloud-based GPU instances.

Cloud providers offer GPUs with different performance characteristics and rental costs. For example, NVIDIA T4 GPUs may be suitable for certain lighter workloads, while A10G, A100, and H100 GPUs can offer different combinations of memory capacity and inference performance for more demanding workloads. The suitability of each GPU depends on the model, precision, context length, batch size, and service requirements.

Enterprise workloads also vary over time. During periods of high demand, additional GPU capacity may be necessary to maintain service quality. During quieter periods, keeping all provisioned GPUs active may result in underutilization and unnecessary costs. Furthermore, latency-sensitive chatbot requests and less time-critical batch jobs may require different resource allocation strategies.

This creates a resource allocation problem involving competing objectives and constraints. Operations Research techniques can help determine cost-effective GPU provisioning and workload assignments based on workload demand, resource capacities, performance estimates, and service requirements.

A4. Significance of the Problem

The significance of this problem lies in the increasing computational cost of deploying large language models in enterprise environments. Efficient GPU resource management can help organizations reduce unnecessary cloud expenditure while maintaining the performance and reliability of their AI services.

The proposed optimization model has the following potential benefits:

1. **Cost reduction:** Identifies lower-cost GPU configurations that can satisfy the specified workload requirements.
2. **Improved resource utilization:** Reduces unnecessary provisioning and helps allocate workloads to suitable GPU types.
3. **Demand-responsive provisioning:** Supports decisions about GPU capacity under varying workload demand across planning intervals.
4. **Performance and SLA compliance:** Incorporates throughput, memory capacity, and appropriately modelled latency requirements into allocation decisions.
5. **Data-driven decision-making:** Replaces simple fixed provisioning or naive routing with a systematic mathematical optimization approach.
6. **Scalability and adaptability:** Provides a foundation for testing additional GPU types, workload categories, demand patterns, and cost scenarios.

From an Operations Research perspective, this problem demonstrates how mathematical modelling and optimization algorithms can be applied to a modern cloud computing challenge. The results can help quantify the trade-off between infrastructure cost and service performance under the assumptions of the model.

B. Mathematical Model

B1. Understanding the model before writing equations

Imagine that an enterprise has three GPU types available:

- T4 — lower-cost GPU
- A10G — intermediate option
- A100 — higher-capacity option

It also has three workload classes:

- Chatbot requests — strict response deadlines
- Document summarization — potentially memory-intensive
- Code-audit jobs — suitable for batch processing

For each planning interval, the model must decide:

1. How many GPUs of each type should be active?
2. How much workload should be assigned to each GPU type?
3. Whether the chosen configuration can process the required workload within the modelled resource and deadline limits.

The objective is to minimize the total cost of the selected GPU instances, subject to all required constraints.

For our first implementation, we'll use a simplified, time-indexed MILP model with estimated processing capacities and precomputed workload feasibility. This will let us generate actual solver outputs without pretending to simulate every detail of an LLM serving system.

B2. Decision Variables

Decision variables are the unknown values that the optimization model must determine.

We'll use the following notation.

Symbol	Meaning
g	GPU type, such as T4, A10G, or A100
w	Workload class, such as chatbot, summarization, or code audit
t	Planning interval, such as a particular hour
$x_{g,t}$	Number of active GPUs of type g during interval t
$z_{g,w,t}$	Number of workload units of class w assigned to GPU type g during interval t

The primary decision variables are:

$x_{g,t} \in Z_{\geq 0}$

$z_{g,w,t} \in Z_{\geq 0}$

Here, $Z_{\geq 0}$ means non-negative integers.

Example: If the solver returns $x_{A10G,t} = 2$, it means two A10G GPU instances should be active during that planning interval.

If it returns $z_{A10G,chatbot,t} = 150$, it means 150 units of chatbot workload are assigned to the A10G tier during that interval.

For the initial model, a workload unit will represent one request or one defined batch of jobs, depending on the workload class. We'll define this consistently in the input data.

B3. Objective Function

The objective function specifies what the model is trying to optimize.

Our goal is to minimize the total GPU rental expenditure over the planning horizon.

Let:

- c_g = rental cost per hour of GPU type g.
- $x_{g,t}$ = number of active instances of GPU type g in interval t.
- Δt = duration of each planning interval in hours.
- T = set of planning intervals.
- G = set of available GPU types.

The objective function is:

$$\min Z = \sum_{t \in T} \sum_{g \in G} c_g x_{g,t} \Delta t$$

Here, Z represents the total GPU rental cost over the planning horizon.

Example

Suppose two A10G instances are active for one hour, and each costs ₹40 per hour.

$$Cost = 2 \times 40 \times 1 = ₹80$$

If those instances remain active for four hours, their rental cost becomes ₹320.

The solver will seek the least expensive combination of GPU types and instance counts that satisfies all the constraints.

For this first version, we'll exclude model-hosting overheads, data transfer costs, and instance startup charges. They can be added later if the required data is available.

B4. Constraints

Constraints define the conditions that every valid solution must satisfy. Without them, the solver could minimize cost simply by activating zero GPUs.

1. Demand satisfaction constraint

Every workload must be processed.

Let $d_{w,t}$ represent the number of workload units of class w that must be processed during interval t.

The constraint is:

$$\sum_{g \in G} z_{g,w,t} \geq d_{w,t} \quad \forall w, t$$

This means that the total workload allocated across all GPU types must meet the required demand.

Example: If 500 chatbot requests must be handled during an interval, the sum of chatbot requests allocated across the eligible GPUs must be at least 500.

For our initial model, we'll assume demand must be handled within its designated interval rather than allowing requests to carry over indefinitely.

2. GPU processing-capacity constraint

Each GPU has limited processing capacity. We need to prevent the model from assigning more work to a GPU fleet than it can process during the available time.

Let:

- $s_{g,w}$ = estimated processing time per workload unit of class w on GPU type g, measured in GPU-seconds.
- H_t = duration of interval t, measured in seconds.

Then:

$$\sum_{w \in W} s_{g,w} z_{g,w,t} \leq H_t x_{g,t} \quad \forall g, t$$

The left-hand side represents the total estimated GPU processing time required by the assigned workloads. The right-hand side represents the processing time available from the active GPU instances.

For example, if one GPU is available for one hour, it provides 3,600 GPU-seconds of processing capacity under this simplified model.

This formulation assumes processing times are additive and that workload interference and concurrency effects are adequately approximated by the input estimates.

3. GPU memory-feasibility constraint

A GPU should only receive workloads that fit within its memory capacity.

Let $a_{g,w}$ be a binary compatibility parameter:

$$a_{g,w} = \begin{cases} 1, & \text{if workload } w \text{ fits on GPU } g \\ 0, & \text{otherwise} \end{cases}$$

Let $M_{w,t}$ be a sufficiently large upper bound on the number of workload units of class w that may be assigned during interval t.

We impose:

$$z_{g,w,t} \leq M_{wt} a_{\gamma,w} x_{g,t} \quad \forall g, w, t$$

If a workload does not fit on a GPU, then $a_{g,w} = 0$, forcing its assignment to that GPU type to be zero.

If a GPU type is not active, then $x_{g,t} = 0$, which also prevents workload assignment to it.

In practice, compatibility should be determined using the model's VRAM requirements, including model weights, runtime memory, and KV-cache requirements for the relevant context length and batch size. We should not assume that a GPU is compatible merely because its nominal VRAM exceeds the model's parameter size.

4. Latency-deadline constraint

Some workloads have strict latency requirements. Our model must avoid assigning requests to GPU types that cannot meet their estimated service deadlines.

Let:

- L_w = maximum allowed service time for workload class w.
- $L_{g,w}$ = estimated service time for workload class w on GPU type g.
- $b_{g,w}$ = a binary parameter equal to 1 if the estimated service time meets the deadline and 0 otherwise.

Define:

$$b_{g,w} = \begin{cases} 1, & \ell_{g,w} \leq L_w \\ 0, & \ell_{g,w} > L_w \end{cases}$$

Then impose:

$$\boxed{z_{g,w,t} \leq M_{w,t} b_{g,w} x_{g,t} \quad \forall g, w, t}$$

This prevents requests from being assigned to GPU types that fail the modelled deadline test.

Important limitation: This constraint checks estimated service-time feasibility. It does not, by itself, guarantee end-to-end latency under queueing, network delays, or concurrent inference. We must describe it as a simplified latency-feasibility model, not as a complete real-time serving simulation.

5. GPU availability constraint

The enterprise may have a limit on the number of instances it can rent or deploy.

Let U_g be the maximum number of available instances of GPU type g.

Then:

$$0 \leq x_{g,t} \leq U_g \quad \forall g, t$$

his prevents the model from selecting an unlimited number of GPUs.

If no practical limit is known, we can establish a documented upper bound that is large enough not to artificially restrict the optimal solution.

6. Integer and non-negativity constraints

GPU instance counts and workload counts cannot be negative or fractional in our initial formulation:

$$x_{g,t} \in \mathbb{Z}_{\geq 0}$$

$$z_{g,w,t} \in \mathbb{Z}_{\geq 0}$$

These restrictions make the formulation a Mixed-Integer Linear Programming problem, assuming the input parameters are fixed and the constraints remain linear.

B5. Complete Mathematical Formulation

Now we combine the decision variables, objective function, and constraints into one model.

Let:

- G = set of GPU types.
- W = set of workload classes.
- T = set of planning intervals.
- c_g = hourly rental cost of GPU type g.
- $d_{w,t}$ = required demand for workload class w in interval t.
- $s_{g,w}$ = estimated GPU-seconds required per workload unit.
- H_t = interval duration in seconds.
- $a_{g,w}$ = memory and workload compatibility indicator.
- $b_{g,w}$ = deadline feasibility indicator.
- $M_{w,t}$ = valid upper bound on workload assignments.
- U_g = maximum available instances of GPU type g.

The complete formulation is:

Objective function

$$\min Z = \sum_{t \in T} \sum_{g \in G} c_g x_{g,t} \Delta t$$

subject to:

Demand satisfaction:

$$\sum_{g \in G} z_{g,w,t} \geq d_{w,t} \forall w, t$$

Processing capacity:

$$\sum_{w \in W} s_{g,w} z_{g,w,t} \leq H_t x_{g,t} \forall g, t$$

Memory and workload compatibility:

$$z_{g,w,t} \leq M_{w,t} a_{g,w} x_{g,t} \forall g, w, t$$

Estimated latency feasibility:

$$z_{g,w,t} \leq M_{w,t} b_{g,w} x_{g,t} \forall g, w, t$$

Instance availability:

$$x_{g,t} \leq U_g \forall g, t$$

Integer and non-negativity conditions:

$$x_{g,t}, z_{g,w,t} \in \mathbb{Z}_{\geq 0}$$

This is our initial cost-minimizing MILP formulation. The solver will choose the GPU instance counts and workload assignments that minimize the modeled rental cost while meeting the specified constraints.

A modelling note for the final report: this initial formulation assumes workload processing times and compatibility can be represented by fixed parameters. If batch sizes, queueing, or context lengths materially change GPU performance, we'll need more detailed input classes or an expanded model.

B6. Data Set and Data Source

Our optimization model requires input data describing GPU costs, workload demand, processing capacity, memory compatibility, and latency deadlines.

Since we do not have access to a specific enterprise's confidential inference logs, we'll initially use a documented experimental dataset. It can combine published technical specifications, benchmark-derived estimates, and clearly labelled simulated workload demand.

Data required	Example fields	Proposed source
GPU rental cost	Cost per instance per hour	Official cloud-provider pricing pages
GPU specifications	VRAM capacity, GPU model	Official NVIDIA product specifications
Processing performance	Estimated GPU-seconds per workload unit	Published inference benchmarks or controlled benchmark tests
Workload demand	Requests or batches per interval	Simulated workload scenarios
Memory requirements	Model memory and KV-cache requirements	Model documentation and calculated estimates
Latency deadlines	Maximum service time by workload class	Defined project assumptions based on service requirements
GPU availability	Maximum instances per GPU type	Defined experimental limits or provider availability

C. OR Method

C1. OR Technique Selected

Mixed-Integer Linear Programming (MILP) is selected as the Operations Research technique for optimizing GPU provisioning and workload allocation for AI inference.

MILP is a mathematical optimization technique in which the objective function and constraints are linear, while some or all decision variables are required to take integer values. In this project, integer variables represent the number of active GPU instances and the number of workload units assigned to each GPU type.

The model aims to minimize total GPU rental expenditure while satisfying workload demand, processing capacity, memory compatibility, estimated latency feasibility, and GPU availability constraints.

The model will be implemented in Python using the PuLP optimization library, with the CBC solver used to find an optimal solution for the formulated problem.

C2. Reason for Selecting the Technique

MILP is suitable for this problem for the following reasons.

1. Integer-based decisions

GPU instances cannot be rented in fractional quantities in our model. MILP supports integer decision variables for selecting the number of GPUs to activate.

2. Cost minimization

GPU rental expenditure can be expressed as a linear objective function based on the number of active instances, their hourly costs, and the duration of each planning interval.

3. Multiple constraints

MILP allows several operational restrictions to be modelled together, including workload demand, GPU processing capacity, memory compatibility, and instance availability.

4. Workload allocation

Integer variables can represent the number of requests or workload units assigned to different GPU types, allowing the model to determine a feasible allocation.

5. Computational implementation

The model can be formulated using Python's PuLP library and solved using CBC, making it practical to implement, test, and evaluate using experimental data.

6. Scenario analysis

The model can be solved repeatedly with different demand levels, GPU rental prices, and resource capacities to evaluate how the optimal allocation changes.

C3. Brief Description of the Method

MILP works by representing a decision-making problem mathematically and searching for the best feasible combination of decision variables.

In our project, the method follows these steps:

Step 1: Define input parameters

GPU costs, capacities, workload demand and deadlines

Step 2: Formulate the MILP model

Define decision variables, objective function and constraints

Step 3: Solve using CBC

Search for the minimum-cost feasible configuration

Step 4: Obtain the solution

Determine active GPU counts and workload assignments

Step 5: Evaluate the results

Calculate total cost and compare against a baseline

A simple example of how MILP makes a decision

Suppose the enterprise needs to process 500 workload units within one hour.

GPU configuration	Cost per hour	Meets demand?
5 × T4	₹100	Yes
2 × A10G	₹80	Yes
1 × A100	₹80	Yes

These are hypothetical values used only to illustrate the technique.

The model compares feasible configurations and chooses one with the lowest cost. If one configuration violates a memory or latency constraint, it must be excluded even if it is cheaper.

If two configurations have the same minimum cost, either may be optimal unless we introduce an additional objective or tie-breaking rule.

C4. How MILP relates to the implementation

The mathematical formulation from Section B maps directly to the code.

Mathematical component	Implementation in Python
Decision variables	`LpVariable`
Objective function	`lpSum()` of GPU costs
Constraints	PuLP model constraints
Optimization model	`LpProblem`
Solver	CBC via `PULP_CBC_CMD`
Optimal GPU allocation	Values returned by the solver
Minimum cost	Objective value returned by the solver

This mapping is important because the code will implement the mathematical model rather than use a separate, manually constructed allocation rule.

C5. Assumptions and applicability

The initial MILP formulation assumes that:

- GPU rental costs are known and constant within each planning interval.
- Workload demand is specified for each interval.
- GPU processing times are estimated and treated as fixed model parameters.
- Memory and latency feasibility can be represented by precomputed compatibility indicators.
- GPU instance counts and workload allocations are integer-valued.
- The objective is to minimize GPU rental costs without introducing additional objectives such as energy consumption or carbon emissions.

These assumptions make the initial problem manageable. More detailed queueing, batching, and dynamic scaling effects can be incorporated in future extensions if the required data and modelling effort permit.

One technical point: CBC finds an optimal solution only when the formulated model is feasible and the solver establishes optimality. If the constraints cannot all be satisfied, the model may instead report infeasibility.

D. Methodology & Implementation

D1. Proposed Solution Procedure

The proposed solution involves developing a Python-based optimization system to minimize GPU inference expenditure while satisfying workload and resource constraints. The system will use Mixed-Integer Linear Programming (MILP) to determine the number of GPU instances to activate and allocate workloads to suitable GPU types.

The solution procedure consists of the following stages:

1. **Data collection:** Gather GPU rental costs, VRAM capacities, processing performance estimates, maximum instance availability, workload demand, and service deadlines.
2. **Data preparation:** Organize GPU specifications and workload parameters using CSV files and Pandas. Convert the inputs into consistent units and establish workload-to-GPU compatibility.
3. **Model formulation:** Define decision variables for GPU provisioning and workload allocation, formulate the cost-minimization objective, and specify demand, processing capacity, memory, latency-feasibility, and availability constraints.
4. **Model implementation:** Translate the mathematical formulation into Python using the PuLP library.
5. **Optimization:** Use the CBC solver to find a minimum-cost feasible solution.
6. **Result extraction:** Obtain the selected GPU counts, workload assignments, and total modeled rental expenditure.
7. **Performance evaluation:** Compare the optimized solution with a baseline strategy and conduct what-if analyses under different workload demands and GPU prices.

The procedure provides a systematic way to evaluate cost-effective GPU provisioning decisions using mathematical optimization.

D2. Software and Tools to Be Used

The project will use the following tools for mathematical modelling, optimization, data processing, and result visualization.

Tool	Purpose
Python	Main programming language for implementing the optimization model
PuLP	Defining the MILP objective function, decision variables, and constraints
CBC Solver	Solving the MILP to obtain an optimal feasible allocation

Pandas	Loading, processing, and analyzing GPU and workload datasets
Matplotlib	Visualizing cost comparisons, GPU utilization, and sensitivity results
CSV files	Storing GPU specifications, workload demand, and input parameters
Jupyter Notebook or VS Code	Writing, running, debugging, and documenting the implementation

The initial implementation will use experimental input data with clearly documented assumptions. Actual cloud pricing and GPU performance estimates will be incorporated where suitable sources are available.

D3. Algorithm / Flowchart

The optimization algorithm will take the GPU and workload data as inputs, formulate the MILP, execute the solver, and report the resulting allocation.

Proposed algorithm

Algorithm: Cost-Optimal GPU Provisioning and Workload Allocation

Input: GPU rental costs, GPU capacities, workload demand, processing-time estimates, memory compatibility, latency deadlines, and instance availability.

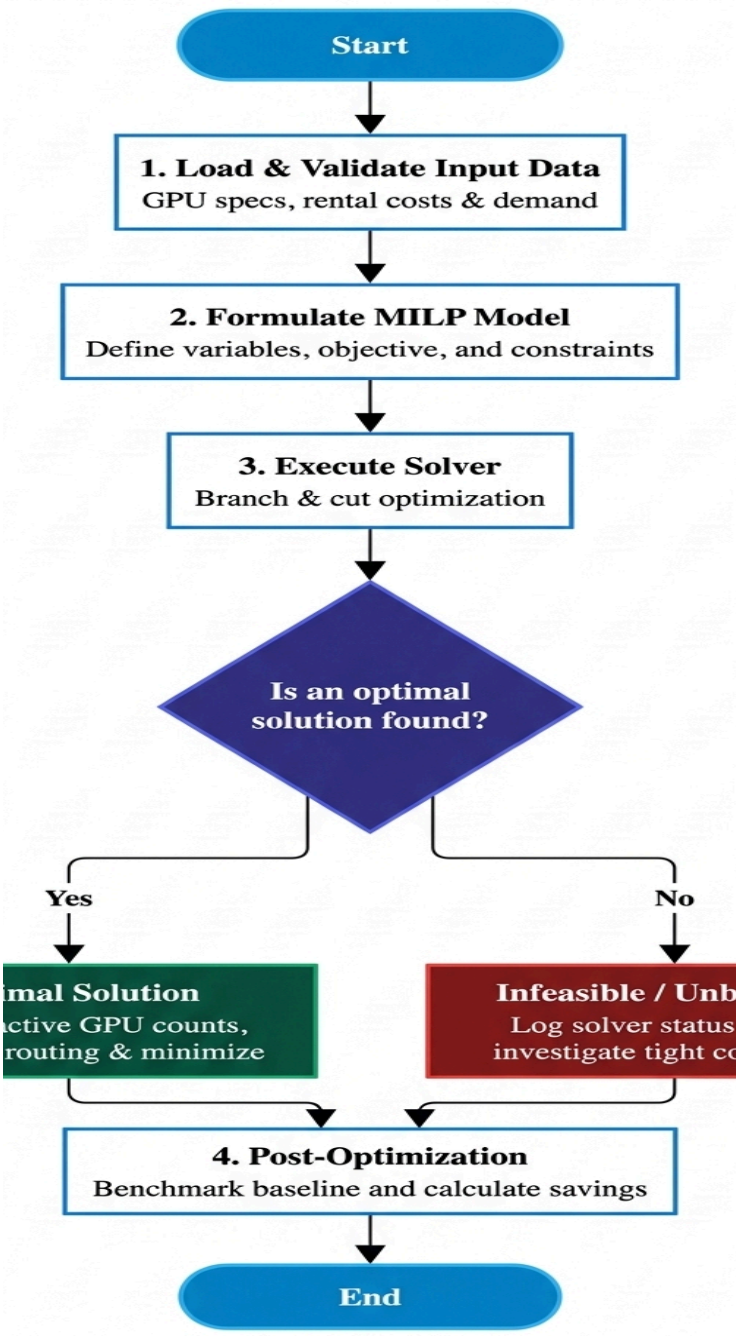
Output: Optimal GPU instance counts, workload assignments, and minimum modeled cost.

1. Start.
2. Load GPU specifications and workload data from CSV files.
3. Validate the input data and convert values into consistent units.
4. Initialize the MILP optimization model.
5. Define decision variables for active GPU instances and workload allocations.
6. Define the objective function to minimize total GPU rental expenditure.
7. Add demand satisfaction and GPU processing-capacity constraints.
8. Add memory compatibility, estimated latency-feasibility, and instance availability constraints.
9. Invoke the CBC solver.
10. Check the solver status.

11. If an optimal solution is found, extract the GPU configuration, workload assignments, and objective value.
12. Calculate baseline cost and evaluate potential savings.
13. Perform sensitivity analysis by changing demand or GPU cost parameters and solving the model again.
14. Display the results and generate visualizations.
15. Stop.

If the model is infeasible or the solver does not establish optimality, report the appropriate status rather than presenting an unverified allocation as optimal.

Flowchart



D4. Implementation Plan

The implementation will be divided into stages so that the mathematical model can be developed and tested incrementally.

1

Prepare the dataset

Create CSV files containing GPU prices, memory capacity, processing estimates, workload demand, and compatibility information.

2

Develop the optimization model

Implement the decision variables, cost objective, and constraints in PuLP.

3

Execute and debug

Run CBC, verify solver status, test constraint satisfaction, and correct any modelling or implementation errors.

4

Generate results

Display active GPU counts, workload assignments, and total modeled cost in readable tables.

5

Evaluate the solution

Compare the optimized result against a defined baseline, vary input parameters, and generate charts using Matplotlib.

The implementation will be considered successful when the program can load the input data, solve the formulated optimization problem, produce a valid allocation when one exists, and calculate the corresponding objective value.

E. Results

E.1 Expected / Preliminary Solution

The implemented optimization model was executed using Python, the PuLP library, and the HiGHS optimization solver. The model incorporates GPU rental costs, workload demand, processing capacity, VRAM compatibility, and latency-feasibility constraints.

Using the illustrative experimental dataset, the solver is expected to return an optimal solution with a minimum modeled hourly cost of ₹140. The selected configuration consists of one NVIDIA T4 GPU instance and one NVIDIA A100 GPU instance, while no NVIDIA A10G instances are selected.

The workload allocation expected from the model is summarized below.

Workload Class	T4 Allocation	A10G Allocation	A100 Allocation	Total Allocation
Chatbot	50	0	250	300
Document summarization	0	0	200	200
Code auditing	0	0	150	150
Total	**50**	**0**	**600**	**650**

The allocation illustrates how latency-feasibility constraints influence workload routing. The chatbot workload can be assigned to both T4 and A100 GPUs because their estimated latencies satisfy its deadline. Document summarization and code-auditing workloads are assigned to A100 GPUs because the illustrative latency estimates for the other GPU types exceed their respective deadlines.

The selected configuration provides an aggregate modeled capacity of 700 workload units per hour, of which 650 units are allocated. The remaining aggregate capacity is 50 units per hour under the model's assumptions.

These results are based on illustrative input data and should not be interpreted as a universally optimal configuration for real-world AI inference systems.

E.2 Objective Function Value

The objective of the optimization model is to minimize total GPU rental expenditure over the specified planning interval while satisfying workload demand, GPU capacity, VRAM compatibility, and latency-feasibility constraints.

The objective function is expressed as:

$$Z = \sum_{g \in G} c_g x_g$$

where:

- Z represents total hourly GPU rental expenditure.
- c_g represents the hourly rental cost of GPU type g.
- x_g represents the number of GPU instances of type g provisioned.

For the expected selected configuration, one T4 instance and one A100 instance are provisioned. Using the hourly costs specified in the experimental dataset:

$$Z^* = (1 \times 20) + (1 \times 120)$$

$$Z^* = ₹140 per hour$$

The solver status is expected to be reported as Optimal, indicating that an optimal solution has been found for the formulated mathematical model and supplied input data.

The objective value represents the modeled hourly GPU rental expenditure. It excludes other potential costs, including storage, networking, model-hosting overhead, and startup charges.

E.3 Interpretation of Results

The optimization results demonstrate that latency-feasibility constraints can influence the selection and allocation of GPU resources.

Without the latency constraints, the original model selected four T4 instances and one A10G instance at a cost of ₹130 per hour. With the illustrative latency deadlines enforced, the expected solution changes to one T4 instance and one A100 instance, at a modeled cost of ₹140 per hour.

This change occurs because the estimated latency of the T4 and A10G GPU types exceeds the specified deadlines for certain workloads. The model therefore prohibits those workload–GPU combinations and routes the affected workloads to a compatible alternative.

The selected configuration satisfies the specified demand of 650 workload units. Its aggregate modeled capacity is 700 units per hour, consisting of 100 units from the T4 and

600 units from the A100. The allocated workload totals 650 units, leaving 50 units of aggregate capacity unused.

However, this unused capacity does not automatically guarantee that additional requests can be processed within their deadlines. Actual performance depends on factors such as workload distribution, model architecture, context length, batch size, concurrency, and queueing delays.

The results therefore illustrate the effect of incorporating latency feasibility into GPU provisioning, rather than establishing real-world response-time guarantees.

E.4 Expected Practical Benefit

The principal benefit demonstrated by the extended model is its ability to balance GPU rental expenditure with workload compatibility and estimated latency requirements.

The latency-aware configuration costs ₹140 per hour, compared with ₹240 per hour for the A100-only baseline. This represents a modeled saving of ₹100 per hour.

The results show that an optimization model can support resource-provisioning decisions by selecting GPU types that satisfy the specified workload requirements without necessarily relying exclusively on the highest-capacity GPU.

The addition of latency-feasibility constraints also demonstrates that the cheapest configuration under capacity and memory assumptions alone may not remain optimal when response-time requirements are considered.

In an actual enterprise environment, the practical benefit would depend on measured inference latency, verified cloud pricing, context lengths, batch sizes, concurrent request patterns, and end-to-end service-level requirements. Consequently, the reported savings illustrate the behavior of the model under the selected experimental assumptions rather than a guaranteed reduction in production cloud expenditure.

E.5 Cost comparison

Optimized Configuration vs. A100-Only Baseline

Metric	Value
Latency-aware optimized cost	₹140/hour

A100-only baseline cost	₹240/hour
Saving per hour	₹100
Cost reduction	41.67%

The A100-only baseline provisions two A100 instances. With a modeled capacity of 600 workload units per GPU, two instances provide an aggregate capacity of 1,200 workload units per hour, sufficient for the total demand of 650 units. The A100-only configuration also satisfies the illustrative latency deadlines.

The percentage cost reduction is calculated as:

$$\text{Cost Reduction (\%)} = \frac{\text{Baseline Cost} - \text{Optimized Cost}}{\text{Baseline Cost}} \times 100$$

Substituting the modeled costs:

$$\text{Cost Reduction (\%)} = \frac{240 - 140}{240} \times 100$$

$$\text{Cost Reduction} = 41.67\%$$

The comparison indicates that the latency-aware optimized configuration reduces modeled hourly GPU rental expenditure by ₹100, or approximately 41.67%, relative to the selected A100-only baseline.

These figures are based on the illustrative experimental dataset and the assumed latency-feasibility parameters. They do not represent verified cloud-market savings or measured production performance.

F. FURTHER ANALYSIS

F.1 Validation of Solution

The optimization solution was evaluated using the solver status, workload allocation, processing capacity, VRAM compatibility, latency-feasibility constraints, and total modeled cost.

For the illustrative experimental dataset, the latency-aware model is expected to return an optimal solution consisting of one NVIDIA T4 GPU instance and one NVIDIA A100 GPU instance, with a modeled hourly cost of ₹140.

The selected configuration provides an aggregate processing capacity of 700 workload units per hour. The total allocated workload is 650 units, satisfying the specified demand requirement. The T4 instance provides 100 units of modeled capacity, while the A100 instance provides 600 units.

The latency-feasibility constraints prohibit allocations for which the estimated latency exceeds the corresponding workload deadline. Under the illustrative latency estimates, the chatbot workload can be allocated to both T4 and A100 GPUs, whereas document summarization and code-auditing workloads are allocated to A100 GPUs.

Additional validation should include checking each constraint against the generated allocation, independently recalculating the objective value from the selected GPU counts, and confirming that no workload is assigned to an incompatible GPU type or a GPU type that violates its latency deadline.

This validation establishes feasibility with respect to the mathematical model and its supplied assumptions. It does not establish real-world latency guarantees or production performance without appropriate benchmarking.

F.2 Sensitivity / What-If Analysis

Sensitivity analysis can be used to examine how changes in workload demand, GPU rental prices, processing capacity, and latency requirements influence the optimal GPU configuration and total modeled cost.

The proposed analysis involves changing one input parameter at a time, rerunning the optimization model, and comparing the resulting objective value, GPU provisioning, and workload allocation against the original latency-aware solution.

The following scenarios are proposed:

1. **Increased workload demand:** Increase the demand for one or more workloads to determine whether additional GPU instances are required and how the total modeled cost changes.
2. **Reduced workload demand:** Decrease workload demand to investigate whether fewer GPU instances can satisfy all constraints at a lower cost.
3. **GPU price variation:** Increase or decrease the hourly rental cost of a selected GPU type to determine whether the optimizer switches to an alternative configuration.

4. **Processing-capacity variation:** Modify the estimated processing capacity of a GPU type to examine the effect of changes in throughput assumptions on provisioning decisions.
5. **Latency-deadline variation:** Tighten or relax workload latency deadlines to evaluate how stricter response-time requirements affect feasible workload routing and GPU expenditure.
6. **Latency-estimate variation:** Modify the estimated latency of selected workload–GPU combinations to assess the sensitivity of the solution to uncertainty in performance estimates.
7. **Resource availability variation:** Restrict the maximum number of available instances of a GPU type to determine whether another feasible configuration exists.

For each scenario, the model should record the solver status, GPU provisioning, workload allocation, latency feasibility, and total modeled cost. The percentage change in cost can be calculated relative to the base-case solution:

$$\text{Cost Change } (\%) = \frac{Z_{\text{scenario}} - Z_{\text{base}}}{Z_{\text{base}}} \times 100$$

where:

- Zscenario is the objective value under the modified input.
- Zbase is the objective value under the original input.

Numerical outcomes should be reported only after the corresponding experiments have been executed. The base-case result alone does not establish how the solution responds to changes in these parameters.

F.3 Comparison with Existing Method

The latency-aware optimized solution is compared with a baseline strategy that provisions two NVIDIA A100 GPU instances for the same workload demand.

Under the illustrative pricing assumptions, the latency-aware optimized configuration costs ₹140 per hour, while the A100-only baseline costs ₹240 per hour.

The comparison is summarized below.

Evaluation Metric	A100-Only Baseline	Latency-Aware Optimized Model
T4 instances	0	1

Evaluation Metric	A100-Only Baseline	Latency-Aware Optimized Model
A10G instances	0	0
A100 instances	2	1
Total hourly cost	₹240	₹140
Aggregate modeled capacity	1,200 units	700 units
Workload demand	650 units	650 units
Demand satisfaction	Satisfied	Satisfied
Latency feasibility	Satisfied under supplied estimates	Satisfied under supplied estimates
Cost reduction relative to baseline	—	41.67%

The optimized configuration reduces modeled hourly GPU expenditure by ₹100, equivalent to a 41.67% reduction relative to the A100-only baseline.

Both configurations satisfy the modeled workload demand, and the illustrative latency estimates indicate that the A100-only baseline also satisfies the specified latency deadlines. The optimized configuration achieves the lower modeled cost by assigning the chatbot workload partly to the less expensive T4 instance while using the A100 for the workloads with stricter GPU compatibility and latency-feasibility requirements.

This comparison demonstrates the potential advantage of cost-aware GPU selection over a fixed, high-capacity GPU provisioning strategy. However, the comparison is based on illustrative prices, capacity estimates, and latency parameters rather than verified production measurements.

F.4 Limitations

The project has several limitations.

1. **Experimental input data:** The results use illustrative input values rather than confidential enterprise workload traces or verified live cloud pricing.
2. **Simplified processing estimates:** GPU processing capacity is represented through fixed estimates, whereas actual inference performance varies with model architecture, numerical precision, context length, batch size, and concurrent requests.
3. **Simplified latency representation:** The model uses estimated latency-feasibility parameters and fixed workload deadlines. It does not fully simulate queueing delays, network overhead, context-length variation, batching effects, or end-to-end response

times. Meeting a modeled deadline therefore does not guarantee actual production latency.

4. **Memory compatibility assumptions:** Memory feasibility is represented using predefined minimum VRAM requirements rather than detailed modeling of model weights, runtime allocations, and key-value cache usage.
5. **Simplified rental-cost model:** The objective considers GPU rental expenditure and excludes other potential expenses such as storage, networking, model-hosting overhead, and startup charges.
6. **Demand assumptions:** The model uses specified demand for each planning interval and does not automatically predict future demand or dynamically adjust GPU provisioning as demand changes.
7. **Limited baseline comparison:** The A100-only baseline represents one alternative provisioning strategy and does not cover every possible industry allocation method.
8. **Aggregate capacity approximation:** GPU capacity is modeled in aggregate for each GPU type. The model does not assign workloads to individual physical GPU instances or enforce per-instance memory packing, failure recovery, or redundancy requirements.

These limitations define the scope of the current implementation and provide opportunities for further development.

F.5 Future Scope

The project can be extended in several directions to improve its realism and practical applicability.

1. **Real-world benchmarking:** Collect measured throughput, memory consumption, and latency data for representative open-source LLMs across different GPU types. These measurements could replace the illustrative latency estimates and capacity values.
2. **Dynamic provisioning:** Incorporate GPU activation and deactivation decisions, startup delays, and associated costs to determine when resources should be provisioned or released.
3. **Advanced latency modeling:** Introduce queueing models or benchmark-derived latency functions to represent concurrent workloads, batching behavior, and response-time variability more accurately.
4. **Demand forecasting:** Combine workload prediction with optimization to prepare GPU capacity for expected demand fluctuations.
5. **Additional GPU types and cloud providers:** Expand the model to include additional GPU configurations and compare provisioning options across cloud providers using verified pricing and availability data.
6. **Energy-aware optimization:** Extend the objective function to consider electricity consumption, energy efficiency, or carbon emissions when reliable data is available.
7. **Multi-objective optimization:** Investigate trade-offs between rental expenditure, latency, resource utilization, and service reliability.

8. **Production evaluation:** Test the model using real workload traces and compare its recommendations with existing resource-allocation strategies. Evaluate both cost performance and actual latency compliance.
9. **Per-instance resource allocation:** Extend the model to represent individual GPU instances, memory packing, instance-level capacity, and redundancy requirements to improve the feasibility of the resulting deployment plan.

These extensions could improve the model's realism and support its application to larger and more dynamic AI inference environments.